Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

in cooperation with

OFFIS

# Design and implementation of a web-based and open-source hero administration for the role-playing game “Das Schwarze Auge” (Backend)

Bachelor thesis
Final presentation

Björn Nowak, 25.05.2023

Erstprüferin:
Prof. Dr.-Ing. Astrid Nieße

Professor for *Digitalized Energy Systems*
*Carl von Ossietzky Universität Oldenburg*
*Faculty II, Department of Computing Science*

Zweitprüfer und Betreuer:
Dr. rer. nat. Martin Tröschel

*Group Manager "Distributed Artifical Intelligence“*
*OFFIS e. V. - Institut für Informatik*
*R&D Division* Energy / *Distributed Artifical Intelligence*

# 1 Introduction
## 1.1 Motivation

**P**en **&** **P**aper
**R**ole-**P**laying **G**ame

“Das Schwarze Auge” (DSA) is Germanys most popular P&P RPG

The problems:



**Time-intensive and error-prone character creation and management for players**

- A complex rulebook
  - Vast amount of rules
  - Different categories of rules for different game phases
  - Countless combinations of character feature forms
- Many rulebook extensions increasing complexity further

# 1 Introduction
## 1.1 Motivation

“Das Schwarze Auge” (DSA) is germanys most popular P&P RPG

The countermeasures:



**Various tools supporting players keeping an overview and automating tasks**

- E.g. character layouts, spread sheets and even programs
- Mainly (almost exclusively) created and published by the community

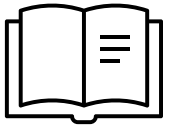
Current restriction <u>which this work tackles</u>:




Non of the popular programs supports the current edition reportingly due to required extensive work and lack of developing capacities

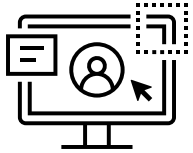
# 1 Introduction
## 1.2 Objectives

Creating a cookbook for formalizing rules of the current (5th) DSA edition, that…



- provides general conventions
- defines a character model
- outlines pre-conditions for processing

Creating a web-based and open-source software as a platform, which…



- implements above cookbook as a proof-of-concept
- enables easy migration to other editions
- supports multiple rulebook extensions
- is very maintainable

Overall focusing on base rules at the character creation/validation



Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

a group of the department for computer science

in cooperation with

OFFIS

# Outline


1 Introduction
- 1.1 Motivation ✓
- 1.2 Objectives ✓
- 1.3 Basics of “Das Schwarze Auge” (DSA)

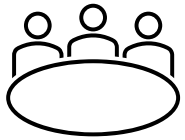
2 Design
- 2.1 Used formalization method
- 2.2 Concept

3 Implementation
- 3.1 Character validation process & core components
- 3.2 Rulebooks logic programs (LPs)

4 Discussion and outlook

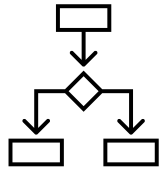
5 Conclusion

# 1 Introduction
## 1.3 Basics of “Das Schwarze Auge” (DSA)



Involved roles:

- One game master
- Characters, also called heros (remaining participants/players)

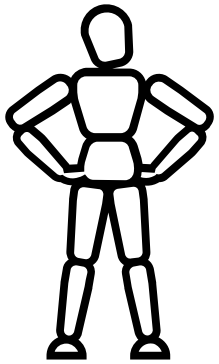


Game process:

- Participants create their character
- The game master leads the characters through a fictional world using a pre-defined rulebook (interactive story-telling)
- Characters may gain adventure points (AP) during a session and use this to advance themselves
- Eat, sleep and repeat

# 1 Introduction
## 1.3 Basics of “Das Schwarze Auge” (DSA)

Simplified specification of a character (by the base rulebook of the 5th edition [1])



| | | |
|---|---|---|
| one | [*R*]ace | of four |
| one | [*C*]ulture | of 25 |
| one | [*P*]rofession | of 33 |
| all | [*CH*]aracteristics | of eight |
| many | [*S*]kills | of 160 |
| some | [*SP*]ecial talents | of 222 |
| some | [*A*]dvantages | of 50 |
| some | [*D*]isadvantages | of 52 |

e.g. **not** including combat techniques or calculated base values

59 talents
45 spells
six rituals
16 cantrips
34 liturgies

$$\binom{R}{1} * \binom{C}{1} * \binom{P}{2} * \binom{CH}{6} * \binom{\frac{S}{4}+\frac{SP}{10}}{20} * \binom{\frac{A}{2}}{10} * \binom{\frac{D}{2}}{5}$$

(equation includes assumptions)

= approx. 3.16078 decillion ($10^{33}$) combinations

# 2 Design

2.1 Used formalization method
2.2 Concept
2.2.1 Conventions
2.2.2 Input model
2.2.3 Preconditions
2.2.4 Cookbook for rule formalization
2.2.5 Result interpretation

# 2 Design
## 2.1 Used formalization method

### Answer Set Programing (ASP)

| | |
|---|---|
| Principal | Finds solutions non-deterministic under constraints / rules |
| Language | Normal-form horn-clauses |
| Techniques | Stable Model Semantics (Answer Set), Grounder, Solver |
| Applications | Constraint-based modelling, Combinatorial optimization, planning problems and decision support |
| Commonness | Wider than Prolog and SAT * |
| Strengths | Precise and complete representation of knowledge, support of negation and uncertainty, complex program flow control |
| Weaknesses | Complex syntax and semantic hence steep learning curve |

* Based on Google Scholar hits using the name and abbreviation

Used framework:

**clingo**
from Potassco

(Potsdam Answer Set Solving Collection)

Used programing language:

**python**

# 2 Design
## 2.1 Used formalization method

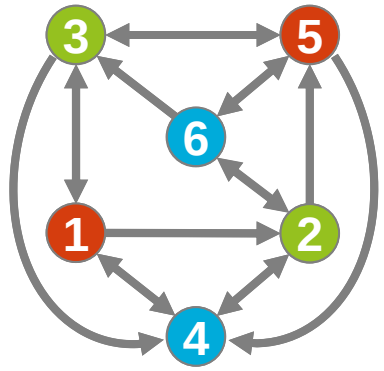
Typical usage / problem: (own representation following [2])
Colorize a directed graph such that no neighbor has the same color.

**Problem instance:**

```
# modelling the graph
node(1..6).
edge(1,2). edge(1,3). edge(1,4).
edge(2,4). edge(2,5). edge(2,6).
edge(3,1). edge(3,4). edge(3,5).
edge(4,1). edge(4,2).
edge(5,3). edge(5,4). edge(5,6).
edge(6,2). edge(6,3). edge(6,5).
# given three colors
color(r). color(g). color(b).
```

**Problem solution:**

```
# one possible answer set
assign(1,r). assign(2,g). assign(3,g).
assign(4,b). assign(5,r). assign(6,b).
```




**Problem encoding:**

```
# assign each node exactly one color
{ assign(N,C) : color(C) } = 1 :- node(N).
# but connected nodes shall not have the same color
:- edge(N,M), assign(N,C), assign(M,C).
```

# 2 Design
## 2.2 Concept / 2.2.2 Input model ∈ Problem instance

Provided by common.lp using the character domain model as context:

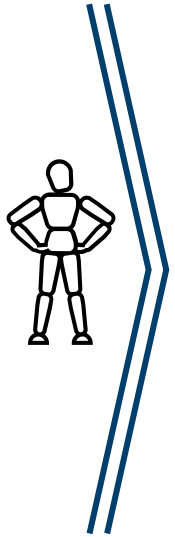
is always loaded

enables interaction between clingo/LP and the application

is api user input

external functions (@) must be deterministic

```
1  #program character_facts.
2  __talent_wrapper(@talents).
3  __combat_technique_wrapper(@combat_techniques).
4  __advantage_wrapper(@advantages).
5  __disadvantage_wrapper(@disadvantages).
6  %% USES := free text reference, can be empty
7  experience_level(@experience_level).
8  race(@race).
9  culture(@culture).
10 profession(@profession).
11 talent(T,LVL) :-
       __talent_wrapper((T,LVL)).
12 combat_technique(CT,LVL) :-
       __combat_technique_wrapper((CT,LVL)).
13 advantage(A,USES,LVL) :-
       __advantage_wrapper ((A,USES,LVL)).
14 disadvantage(DA,USES,LVL) :-
       __disadvantage_wrapper((DA,USES,LVL)).
```



```
experience_level("Average").
race("Elfen").
culture("Auelfen").
profession("Söldner").
talent("Kraftakt",3).  talent("Zechen",3).

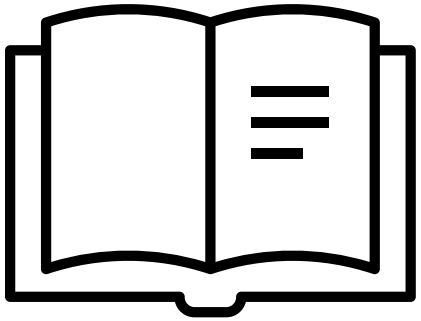
combat_technique("Armbrüste",10).

advantage("Begabung","Singen",1).

disadvantage("Wahrer Name","",1).
```

# 2 Design
## 2.2 Concept / 2.2.4 Cookbook for rule formalization

Three groups of rules based on application-range and extent of compliance:



[GR] Rulebook-unspecific hard rules
- General cross rulebook rules
- Must be followed always

[HR] Rulebook-specific hard rules
- Must be followed always

[SR] Rulebook-specific soft rules
- Non-conformance possible on game master approval

provided by common.lp

provided by each rulebook

# 2 Design
## 2.2 Concept / 2.2.4 Cookbook for rule formalization

[HR.3] A feature form requires <u>one</u> form on a minimum level of another multi-form feature

**Problem instance #4:** (world fact provided by rulebooks)

```
1  requires(profession("Söldner"),talent("Kriegskunst",6)).
```

**Problem encoding #4:** (world rule provided by common.lp)

```
1  %% logical 'or' build with '#count': feature is missing or min. level is not reached
2  missing_level(profession(P),talent(T,MIN_LVL)) :-
       profession(P),
       requires(profession(P),talent(T,MIN_LVL)),
       1 = #count{ x: not talent(T,_) ; x: talent(T,LVL), LVL < MIN_LVL }.
```

```
1 = #count{   ;   }.      1 = 0.      #false
1 = #count{ x ;   }.      1 = 1.      #true
1 = #count{   ; x }.      1 = 1.      #true
1 = #count{ x ; x }.      1 = 1.      #true
```

time

# 2 Design
## 2.2 Concept / 2.2.4 Cookbook for rule formalization

[HR.4] A feature form requires a choice of forms of another multi-form feature on a min level

**Problem instance #5:** (world fact provided by rulebooks)

```
1  %% 'Söldner‘ requires two CTs of a list on at least level 10
2  requires(
       profession("Söldner"),
       any_of(2,combat_technique,("Hiebwaffen","Schwerter","Stangenwaffen"),10)
   ).
```

**Problem encoding #5:** (world rule provided by common.lp)

```
1  missing_level(profession(P),combat_technique(any_of(CHOICES,CTs),MIN_LVL))  :-
       profession(P),
       requires(profession(P),any_of(CHOICES,combat_technique,CTs,MIN_LVL)),
       CHOICES > @count_by("combat_techniques",CTs,MIN_LVL).
```

time

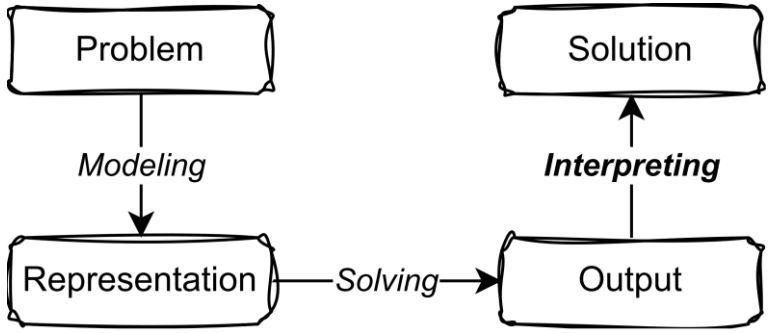
# 2 Design
## 2.2 Concept / 2.2.4 Cookbook for rule formalization

[HR.4] A feature form requires a choice of forms of another multi-form feature on a min level

```
1  from clingo import Symbol, Number
2  ## method of context class (input model) having the character as class field (self.hero)
3  def count_by(self, feature: Symbol, options: Symbol, min_lvl: Symbol) -> Symbol:
4    """
5    :return: count of 'options' (feature forms) of 'feature' having minimum level
6    """
7    ## dynamically get class field (with 'getattr') instead of manual mapping with a switch-case
8    ##   requires 'feature.string' to be exactly the field name of the actual hero model
9    values_lvl: dict[str, int] = {fv.name: fv.level for fv in getattr(self.hero, feature.string)}
10   ## count all features having the minimum level
11   passed = sum(1 for opt in options.arguments if min_lvl.number <= values_lvl.get(opt.string, 0))
12   return Number(passed)
```

# 2 Design
## 2.2 Concept / 2.2.5 Result interpretation

General process: (own representation following [2])




Unusual for ASP:

- Not searching for all possible answer sets
- All problems (LPs) are satisfiable, enabling to…
- …always produce “fixed” result facts

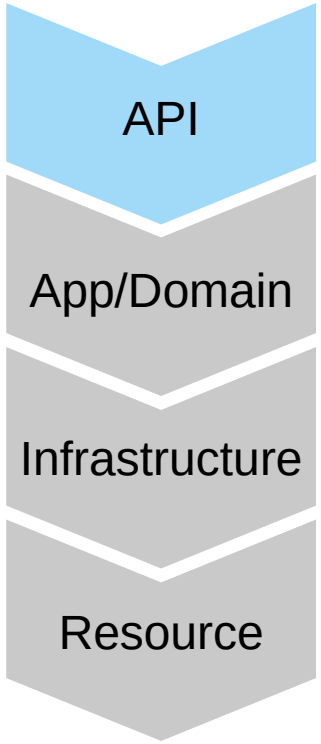
  e.g. problem encoding #4:

```
2  missing_level(profession(P),talent(T,MIN_LVL)) :-
       profession(P),
       requires(profession(P),talent(T,MIN_LVL)),
       1 = #count{ x: not talent(T,_) ; x: talent(T,LVL), LVL < MIN_LVL }.
```

# 3 Implementation

~~3.0 Architecture and design decisions~~
3.1 Character validation process & core components
3.2 Rulebooks logic programs (LPs)
~~3.3 Quality assurance~~
~~3.4 Encounters difficulties~~

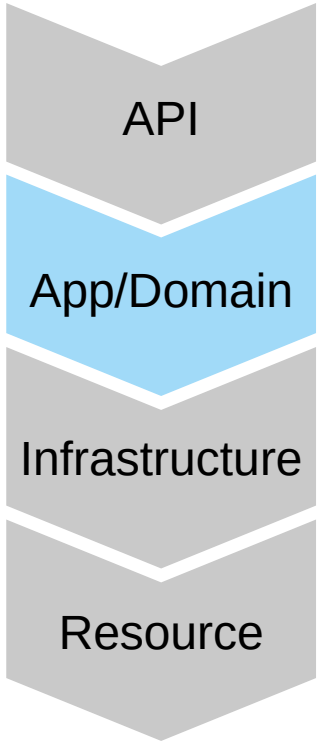
# 3 Implementation
## 3.1 Character validation process & core components




**Example request** (json): `<host>/api/hero/validate?rulebooks=dsa5`

```
{
  "name": "UncleBob",
  "experience_level": "Legendary",
  "race": "Elfen",
  "culture": "Auelfen",
  "profession": "Söldner",
  "talents": {
    "Körperbeherrschung": 2,
    "Kraftakt": 3,
    "Selbstbeherrschung": 4,
    "Zechen": 5,
    "Menschenkenntnis": 3,
    "Überreden": 3,
    "Orientierung": 4,
    "Wildnisleben": 3,
    "Götter & Kulte": 3,
    "Kriegskunst": 6,
    "Sagen & Legenden": 5,
    "Handel": 3,
    "Heilkunde Wunden": 4
  },
  "combat_techniques": {
    "Armbrüste": 10,
    "Raufen": 10,
    "Stangenwaffen": 9,
    "Zweihandschwerter": 10
  },
  "advantages": [
    ["Begabung", "Singen", 1],
    ["Begabung", "Musizieren", 1],
    ["Beidhändig", "", 1],
    ["Dunkelsicht", "", 2]
  ],
  "disadvantages": [
    ["Körpergebundene Kraft", "", 1],
    ["Lästige Mindergeister", "", 1],
    ["Wahrer Name", "", 1]
  ]
}
```

# 3 Implementation
## 3.1 Character validation process & core components

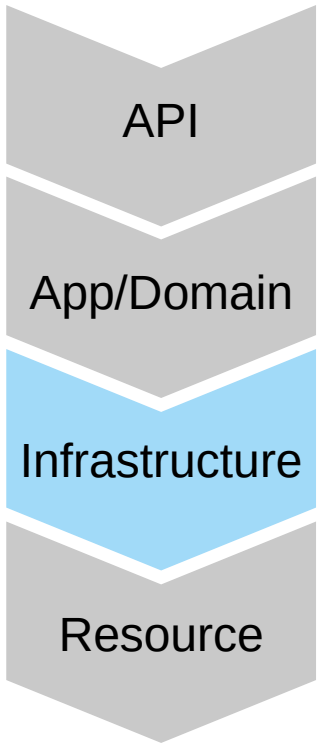



**Engine**:

- Defines which LP files are to be loaded
- Defines which LP sub-programs are to be executed
- Fails whenever given rulebook set is not usable
- Executes character validation step-wise (each one is a sub-program)
  - Default: pre-conditions > feature usable > feature requirements >
  - Detects addition steps within rulebooks LPs

for each feature;
fixed order
race > culture > …

# 3 Implementation
## 3.1 Character validation process & core components

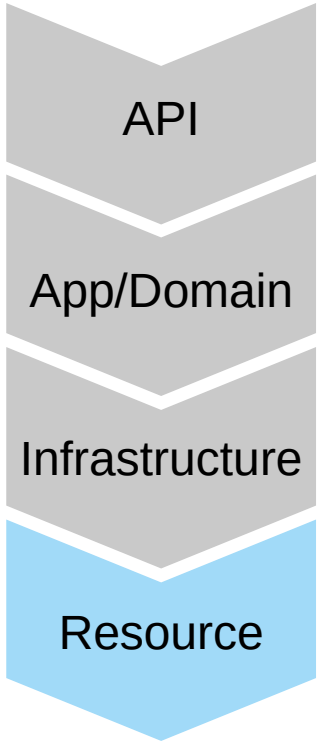



Engine

**ClingoExecutor**:

- Interface between application and ASP framework (loose coupling)
- Loads, ground and execute given LPs and sub-programs
  using the clingo api
- Fails whenever run was not satisfiable
- Accepts an optional context
  - Uses a convenient wrapper for the character model
    convert python datatypes to clingo symbols
- Returns whole model (all symbols; not just an answer set)

# 3 Implementation
## 3.1 Character validation process & core components




ClingoExecutor

**Rulebook LPs**:

– Each rulebook is an own set of LPs

– Contained in separate folders

– Required to have an “_entrypoint.lp” top-level file

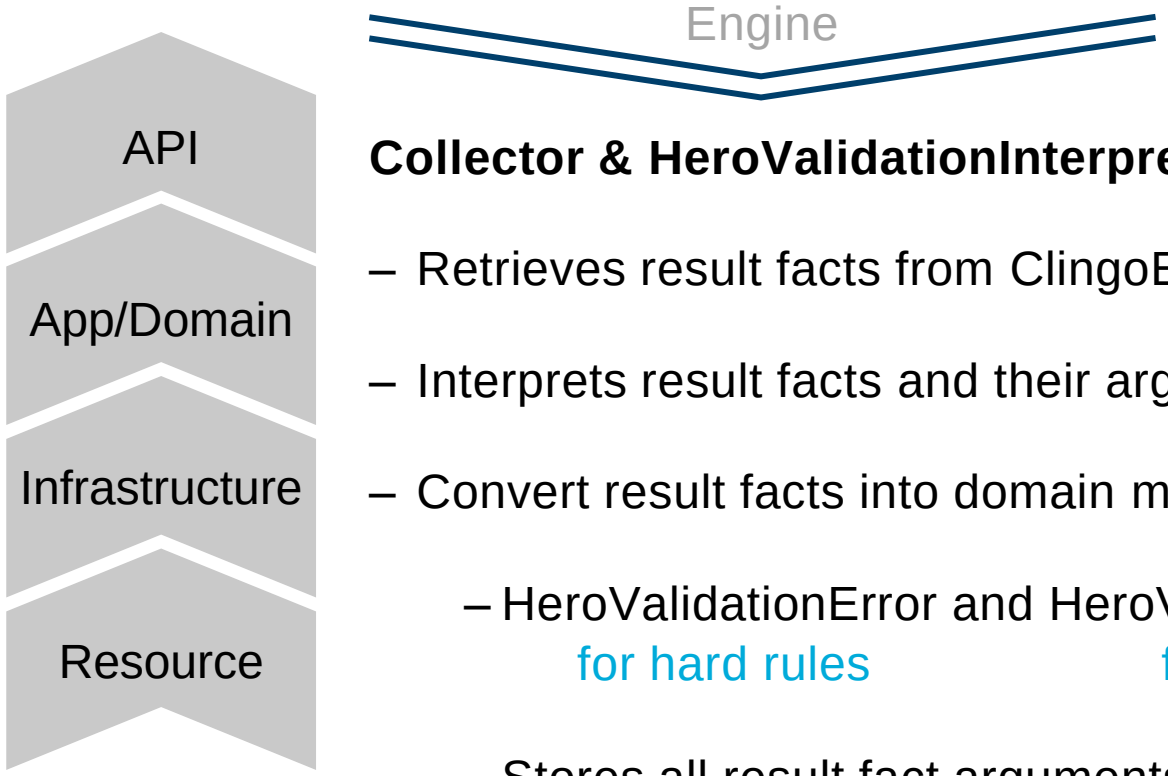
the single file of a rulebook directly used by the engine;
an LP can extend itself by clingo directive “#include”

– Required to have specific sub-programs (used by the Engine)

don’t forget the common.lp

and remember:
rulebook ≈ rulebook (extension)

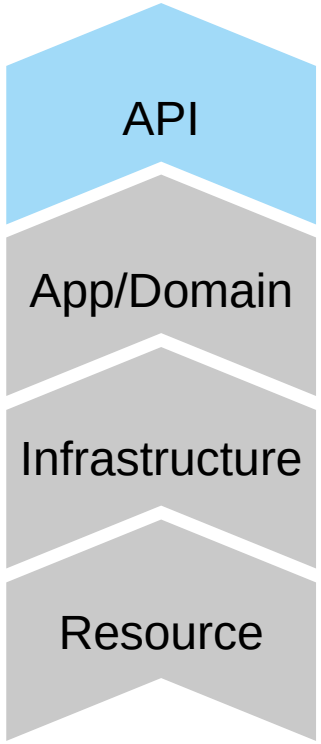
# 3 Implementation
## 3.1 Character validation process & core components




**Collector & HeroValidationInterpreter**:

- Retrieves result facts from ClingoExecutor model (answer set)
- Interprets result facts and their arguments
- Convert result facts into domain models
  - HeroValidationError and HeroValidationWarning
    for hard rules for soft rules
  - Stores all result fact arguments in a map of constant parameters
    enables API users (like a frontend) further processing

# 3 Implementation
## 3.1 Character validation process & core components

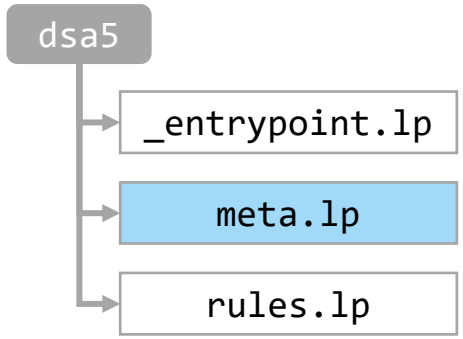



**Example response** (json): `<host>/api/hero/validate?rulebooks=dsa5`

```
{
  "valid": false,
  "errors": [
    {
      "type": "missing_level",
      "message": "Heros 'profession' is missing minimum level '3' for 'talent' of 'Körperbeherrschung'.",
      "parameter": {
        "caused_feature": "profession",
        "caused_feature_value": "Söldner",
        "referred_feature": "talent",
        "referred_feature_value": "Körperbeherrschung",
        "min_level": 3
      }
    }
  ],
  "warnings": []
}
```

# 3 Implementation
## 3.2 Rulebooks logic programs (LPs)

File structure:

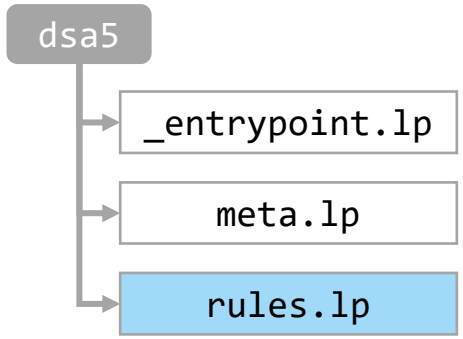



File content:

```
1  #program rulebook_usable.
2  program("rulebook_usable").
3  %% make presence of this rulebook available for others
4  rulebook("dsa5").
5  %% this rulebook depends on nothing, else
6  %%   rulebook_depends("dsa5_extension","dsa5").
7
8  #program world_facts.
9  program("world_facts").
10 known_experience_level("Unerfahren",       12, 10,  8, 95,  8, 0 ).
11 known_experience_level("Durchschnittlich", 13, 10, 10, 98, 10, 1 ).
12 known_race("Elfen"; […]).
13 known_culture("Auelfen"; […]).
14 known_advantage("Begabung",("Singen";"Musizieren"),1).
15 known_disadvantage(("Prinzipientreue";"Verpflichtungen"),"",(1..3)).
16 known_talent("Zechen";"Kraftakt";"Überreden"; […]).
17 known_combat_technique("Armbrüste";"Raufen";"Hiebwaffen"; […]).
18 […]
19 %% pre-conditions
20 max_count(race,1).
21 max_count(culture,1).
22 max_count(profession,1).
```

# 3 Implementation
## 3.2 Rulebooks logic programs (LPs)

File structure:



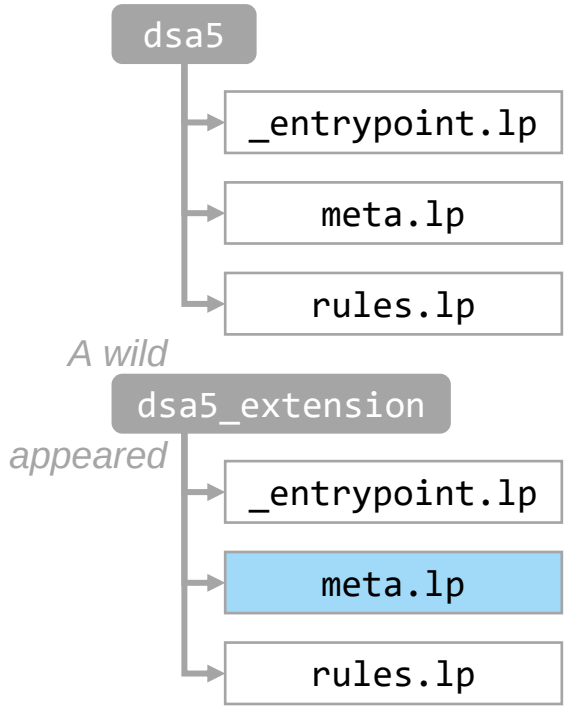

File content:

```
1  #program world_facts.
2  program("world_facts").
3  %% race Elfen
4  requires(race("Elfen"),advantage("Zauberer","",1;"Zweistimmiger Gesang","",1)).
5  has_usual(race("Elfen"),culture("Auelfen";"Firnelfen";"Waldelfen")).
6  has_typical(race("Elfen"),advantage("Begabung",("Singen";"Musizieren"),1; […]).
7  has_atypical(race("Elfen"),disadvantage("Blutrausch","",1; […]).
8  %% race Halbelfen
9  […]
10 %% culture to race
11 requires(culture("Auelfen";"Firnelfen"; […]),race("Elfen")).
12 […]
13 %% culture Auelfen
14 has_usual(culture("Auelfen"),profession("Wildnisläufer";"Zauberweber")).
15 […]
16 %% profession Söldner
17 requires(profession("Söldner"),talent("Körperbeherrschung",3; […]).
18 requires(profession("Söldner"),combat_technique("Armbrüste",10;"Raufen",10)).
19 requires(profession("Söldner"),any_of(1,combat_technique,("Hiebwaffen", […]),10)).
20 […]
```

time

# 3 Implementation
## 3.2 Rulebooks logic programs (LPs)

File structure:




File content:

```
1  #program rulebook_usable.
2  program("rulebook_usable").
3  %% make presence of this rulebook available for others
4  rulebook("dsa5_extension").
5  %% this rulebook depends on base ruelbook dsa5
6  rulebook_depends("dsa5_extension","dsa5").
7
8  #program world_facts.
9  program("world_facts").
10 known_culture("Fjarninger";"Mhanadistani").
11 known_profession("Skuldrun";"Säbeltänzerin").
12 […]
```

Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

a group of the department for computer science

in cooperation with

OFFIS

# 5 Conclusion

# 5 Conclusion

Formalizing the DSA rulebook is possible

…and easy to implement and understand

…but future will show if this holds when

…all rules and more rulebooks are implemented

…and especially character management.

Besides some (intentional) easy-to-fix shortcomings the software

…is maintainable and extensible

…and could even support other DSA editions

hence it is a good starting point for the community.

May soon available for contributions at:
https://github.com/bjoern-nowak/dsa-heldenbogen

# 6 Literature

[1] Ulisses Spiele GmbH: „DSA Regel Wiki“. Aus Unterseiten. 2023. url: https://www.ulisses-regelwiki.de/start.html

[2] Martin Gebser and others: “Answer Set Solving in Practice”. December 2012. doi:10.2200/S00457ED1V01Y201211AIM019:

skipped

# 4 Discussion and outlook

skipped

# 4 Discussion and outlook
# A pre-discussion

[DC.2] Features could be grouped to reduce repetition in rules

(was yet not required due to few groupable features)

```
1  %% Example of a repetition
2  missing_min_lvl(talent(T,LVL),MIN) :- talent(T,LVL), MIN = 0, LVL < MIN .
3  missing_min_lvl(combat_technique(CT,LVL),MIN) :- combat_technique(CT,LVL), MIN = 0, LVL < MIN .
4  %% Group option A: additionally to character facts
5  skill(talent,NAME,LVL) :- talent(NAME,LVL).
6  skill(combat_technique,NAME,LVL) :- combat_technique(NAME,LVL).
7  %% Group option B: replace character facts (not recommended)
8  skill(talent,NAME,LVL) :- __talent_wrapper((NAME,LVL)).
9  skill(combat_technique,NAME,LVL) :- __combat_technique_wrapper((NAME,LVL)).
10 %% Result: single rule possible
11 missing_min_lvl(FEATURE,NAME,LVL,MIN) :- skill(FEATURE,NAME,LVL), MIN = 0, LVL < MIN.
```

skipped

# 4 Discussion and outlook
# A pre-discussion

[DC.4] The "`@count_by`" could not be formalized so that it is easy to understand and maintain

(no real issue: external function only implements simple logic)

```
1  %%%% 'prof := profession' und 'combatT := combat_technique'
2  %% Choice list with three elements having the same key
3  requires(prof("Söldner"),any_of("uniqueID",1,combatT("Hiebwaffen"; […]),10)).
4  %% Approach using "head  aggregation": creates three result facts with same key
       in order to compose a choice list on result interpretation
5  CHOICES > #count{
       KEY,CT
           : missing_level(prof(P),any_of(KEY,CHOICES,combatT,CT),MIN_LVL))
           : requires(prof(P),any_of(KEY,CHOICES,combatT(CT),MIN_LVL)),
               combatT(CT,LVL),
               LVL >= MIN_LVL
   } :-
       prof(P),
       requires(prof(P),any_of(KEY,CHOICES,combatT(_),MIN_LVL)).
```

approach implementation is not a valid LP and hence did not work (yet)

skipped

# 4 Discussion and outlook
## An outlook

[OL.1] There shall be a technical validation (rule) that let rulebook LPs only use known feature forms (in respect of their dependencies)

[OL.2] Character validation errors and warnings shall also state (if possible) what would be right (like which feature forms are available under selected “higher” features)

[OL.3] Character validation warnings shall be ignorable as game master approved

[OL.4] Character management: leveling up using APs

- Keep track of received and spent APs
- Successive validation of each change (“multi-shot solving” ?)

and more like user registration and storing characters in a database